Task 1: Loading Trading Data and Computing Candlestick Data

(i) Suitable explanation

In this task, I added a candlestick summary view for a selected product to understand price behavior without having to read every single order row. A candlestick compresses many prices in a time period into four values:

Open = first price in that period

High = highest price in that period

Low = lowest price in that period

Close = last price in that period

This gives a quick picture of direction (open to close) and volatility (high vs low).

ASK and BID candlesticks are displayed separately because they represent different sides of the market.

One small but important improvement I made is output precision. I print OHLC using fixed 10 decimal places in Candlestick::displayRow(). Some prices may differ by very small amounts, and with fewer decimals, and initially those differences got rounded away and made candles look "identical" even when they were not.

Also, the provided dataset (20200601.csv) is effectively one trading day, lots of timestamps but the same calendar day. So, if I only rely on the raw CSV, the daily, monthly and yearly summary naturally produces just one row for that day. The multi-day daily rows appear come from simulated trades, and I discuss that under Task 4.

(ii) Implementation logic discussion + pseudocode fragments

Where I implemented it (files/classes)

I separated the feature into UI flow vs calculation:

MerkelMain::showCandlesticks()
Handles product selection, menu options (daily/monthly/yearly/range), and printing the tables.

CandlestickCalculator (computeDailyCandles / computeMonthlyCandles / computeYearlyCandles)
Does the actual grouping and OHLC calculation.

/CandlestickCalculator.cpp

```cpp
std::vector<Candlestick> CandlestickCalculator::computeDailyCandles(const std::vector<OrderBookEntry> &entries)
{
    std::map<std::string, std::vector<OrderBookEntry>> grouped;

    for (auto &e : entries)
    {
        std::string day = e.timestamp.substr(0, 10);
        // Normalize separators so grouping keys are consistent even if timestamps contain '/'
        std::replace(day.begin(), day.end(), '/', '-');
        grouped[day].push_back(e);
    }

    std::vector<Candlestick> candles;

    for (auto &pair : grouped)
    {
        // List order determines Open/Close, sort by timestamp before using front()/back() if needed
        auto &list = pair.second;
        std::sort(list.begin(), list.end(),
                  [](const OrderBookEntry &a, const OrderBookEntry &b)
                  {
                      return a.timestamp < b.timestamp;
                  });

        double open = list.front().price;
        double close = list.back().price;
        double high = list.front().price;
        double low = list.front().price;

        // Scan all entries in this timeframe to find the highest and lowest traded prices
        for (auto &e : list)
        {
            if (e.price > high)
                high = e.price;
            if (e.price < low)
                low = e.price;
        }

        candles.emplace_back(pair.first, open, high, low, close);
    }
```

Candlestick.cpp/.h

Stores date, open, high, low, close and prints a row using consistent formatting (10dp).

DateUtils.cpp/.h

Validates user input dates (YYYY-MM-DD), extracts day from timestamps, and checks whether a date is within a range.

Step A: Product selection and splitting into asks/bids

In showCandlesticks(), I first show all known products and let the user choose one. Then I gather all entries and split them into two lists based on OrderBookType:

- if entry is an ask, it goes into asks
- if entry is a bid, it goes into bids

This step is what allows me to print ASK candles and BID candles separately.

Step B: Grouping entries into time buckets and computing OHLC

The calculator groups entries by a time key depending on the summary:

1. **Daily key:** first 10 characters of timestamp (day), normalized to YYYY-MM-DD
2. **Monthly key:** first 7 characters of timestamp (month), normalized to YYYY-MM
3. **Yearly key:** first 4 characters of timestamp (year), YYYY

For daily and monthly, I replace / with - in the grouping key so the printed keys are consistent even if timestamps contain /. After grouping, I sort each bucket by timestamp and compute:

Open from the first entry's price

Close from the last entry's price

High/Low by scanning all prices in that bucket

Step C: Default yearly view, then an additional options menu

After the user selects a product, I immediately show the Yearly summary as default. Then I show the "Additional Options" menu so the user can switch to Daily, Monthly, Yearly again, or a Date Range view, and Change Product. And instead of having the user enter a product combination, they're shown a list of available products as options to choose from to display their respective candlestick summary.

Step D: Date range filter (validated input + filtered candles)

For the range filter option, I first compute the minimum and maximum available day from the data so the user can see what range exists. Then I repeatedly ask for input until it is valid:

1. Date format must be YYYY-MM-DD
2. End date must be ≥ start date
3. Both dates must be within the available min/max

After that, I filter asks and bids separately using DateUtils::isWithinRange() and generate daily candles for only the filtered subset (daily is the most readable for a range view).

```
}

std::vector<OrderBookEntry> fA, fB;

for (auto &e : asks)
{
    std::string d = DateUtils::extractDay(e.timestamp);
    if (DateUtils::isWithinRange(d, start, end))
        fA.push_back(e);
}

for (auto &e : bids)
{
    std::string d = DateUtils::extractDay(e.timestamp);
    if (DateUtils::isWithinRange(d, start, end))
        fB.push_back(e);
}

auto fdA = CandlestickCalculator::computeDailyCandles(fA);
auto fdB = CandlestickCalculator::computeDailyCandles(fB);

UI::printTitle("FILTERED RANGE (" + product + ")");

std::cout << "ASK (Filtered)" << std::endl;
UI::printDivider();

std::cout << std::left
          << std::setw(14) << "Date"
          << std::setw(18) << "Open"
          << std::setw(18) << "High"
          << std::setw(18) << "Low"
          << std::setw(18) << "Close"
          << std::endl;

for (auto &c : fdA)
    c.displayRow();
```

Pseudocode 1 - Candlestick computation (daily/monthly/yearly)

```
COMPUTE-CANDLES(entries, KEY_FN):
  grouped ← map from string to list
  FOR each entry IN entries:
    key ← KEY_FN(entry.timestamp)
    key ← REPLACE(key, "/", "-")        // normalize separators
    grouped[key].append(entry)
  candles ← empty list
  FOR each key IN grouped (ascending):
    list ← grouped[key]
    SORT list by timestamp ascending
    open  ← list[0].price
    close ← list[last].price
    high  ← open
    low   ← open
    FOR each e IN list:
      high ← max(high, e.price)
      low  ← min(low, e.price)
    candles.append( (key, open, high, low, close) )
  RETURN candles
```

Pseudocode 2 - Candlestick menu flow

```
SHOW-CANDLESTICKS():
  LOOP:
    product ← user selects from known products
    allEntries ← orderBook.getAllOrders()
```

```
    asks ← entries in allEntries where product matches and type=ASK
    bids ← entries in allEntries where product matches and type=BID
    DISPLAY yearly candles for asks and bids
    LOOP:
      choice ← user selects (Daily/Monthly/Yearly/Range/Change/Exit)
      IF Daily:   DISPLAY daily(asks), daily(bids)
      IF Monthly: DISPLAY monthly(asks), monthly(bids)
      IF Yearly:  DISPLAY yearly(asks), yearly(bids)
      IF Range:
        start,end ← read + validate dates
        fA ← asks within [start,end]
        fB ← bids within [start,end]
        DISPLAY daily(fA), daily(fB)
      IF Change: BREAK to pick another product
      IF Exit:   RETURN
```

Pseudocode 3 - Date range validation logic

```
READ-VALID-RANGE(minDate, maxDate):
  LOOP:
    start ← input "YYYY-MM-DD"
    end   ← input "YYYY-MM-DD"
    IF not validFormat(start) OR not validFormat(end):
      print error; CONTINUE
    IF end < start:
      print error; CONTINUE
    IF start < minDate OR end > maxDate:
```

print error; CONTINUE

RETURN (start, end)

(iii) Screenshots and evidence of the final output

Main Menu > Choose option 7 to view CandleStick Summary

```
==================== WELCOME TO MERKLEREX ====================
==================== MERKLEREX MAIN MENU ====================
1: Print help
2: Print exchange stats
3: Make an offer (ask)
4: Make a bid
5: Print wallet
6: Continue to next TimeFrame
7: Show Candlestick summary
8: Wallet & Transactions Menu
9: Simulate trading activity
10: Exit program
------------------------------------------------------------
Current time is: 2020/06/01 11:57:30
------------------------------------------------------------
Enter choice (1-10):
7
==================== AVAILABLE PRODUCTS ====================
1: BTC/USDT
2: DOGE/BTC
3: DOGE/USDT
4: ETH/BTC
5: ETH/USDT
------------------------------------------------------------
Enter choice (1-5):
```

Default Yearly Summary > Choose option 1 to view Daily Summary

```
------------------------------------------------------------
Enter choice (1-5):
1
================= YEARLY SUMMARY (BTC/USDT) =================
ASK Candlesticks (Yearly)
------------------------------------------------------------
Year          Open              High              Low               Close
2020          9549.3999982900   9614.9238645400   9495.9293918700   9600.0000000000

BID Candlesticks (Yearly)
------------------------------------------------------------
Year          Open              High              Low               Close
2020          9503.4480656300   9588.5205724100   9452.9401565700   9542.8826887600
------------------------------------------------------------
Additional Options:
1. View Daily Summary
2. View Monthly Summary
3. View Yearly Summary
4. View Summary by Date Range
5. Change Product
6. Exit Candlestick View
------------------------------------------------------------
Enter choice (1-6):
1
================= DAILY SUMMARY (BTC/USDT) =================
ASK (Daily)
------------------------------------------------------------
Date          Open              High              Low               Close
2020-06-01    9549.3999982900   9614.9238645400   9495.9293918700   9600.0000000000

BID (Daily)
------------------------------------------------------------
Date          Open              High              Low               Close
2020-06-01    9503.4480656300   9588.5205724100   9452.9401565700   9542.8826887600
------------------------------------------------------------
```

Choose option 2 to view Monthly Summary

```
------------------------------------------------------------
Additional Options:
1. View Daily Summary
2. View Monthly Summary
3. View Yearly Summary
4. View Summary by Date Range
5. Change Product
6. Exit Candlestick View
------------------------------------------------------------
Enter choice (1-6):
2
================= MONTHLY SUMMARY (BTC/USDT) =================
ASK (Monthly)
------------------------------------------------------------
Month         Open              High              Low               Close
2020-06       9549.3999982900   9614.9238645400   9495.9293918700   9600.0000000000

BID (Monthly)
------------------------------------------------------------
Month         Open              High              Low               Close
2020-06       9503.4480656300   9588.5205724100   9452.9401565700   9542.8826887600
------------------------------------------------------------
```

Choose option 4 to view Summary by Date Range

```
------------------------------------------------------------
Additional Options:
1. View Daily Summary
2. View Monthly Summary
3. View Yearly Summary
4. View Summary by Date Range
5. Change Product
6. Exit Candlestick View
------------------------------------------------------------
Enter choice (1-6):
4
==================== DATE RANGE FILTER ====================
Available range: [2020-06-01 - 2020-06-01]
------------------------------------------------------------
Enter start date (YYYY-MM-DD):
2020-06-01
Enter end date (YYYY-MM-DD):
2020-06-01
================ FILTERED RANGE (BTC/USDT) =================
ASK (Filtered)
------------------------------------------------------------
Date          Open              High              Low               Close
2020-06-01    9549.3999982900   9614.9238645400   9495.9293918700   9600.0000000000

BID (Filtered)
------------------------------------------------------------
Date          Open              High              Low               Close
2020-06-01    9503.4480656300   9588.5205724100   9452.9401565700   9542.8826887600
------------------------------------------------------------
```

Choose option 4 to view Candlestick summary of a different product

```
Additional Options:
1. View Daily Summary
2. View Monthly Summary
3. View Yearly Summary
4. View Summary by Date Range
5. Change Product
6. Exit Candlestick View
------------------------------------------------------------
Enter choice (1-6):
5
==================== AVAILABLE PRODUCTS ====================
1: BTC/USDT
2: DOGE/BTC
3: DOGE/USDT
4: ETH/BTC
5: ETH/USDT
------------------------------------------------------------
Enter choice (1-5):
4
================= YEARLY SUMMARY (ETH/BTC) =================
ASK Candlesticks (Yearly)
------------------------------------------------------------
Year          Open              High              Low               Close
2020          0.0251094500      0.0252832500      0.0247586200      0.0251611100

BID Candlesticks (Yearly)
------------------------------------------------------------
Year          Open              High              Low               Close
2020          0.0246194800      0.0250842500      0.0245344800      0.0249468600
------------------------------------------------------------
```

TASK 2: Login or Register a Trading User Profile

(i) Suitable explanation

For this task, I added a user account system, so the program is not one shared trader, but instead works like a real platform where each person has their own identity and wallet.

When the program is run, the user lands on a Login Menu where they can either:

Login using a username + password

Register as a new user (if they don't already exist)

Recover my username (if user forgot it)

Reset my password (if user forgot it)

It is intentionally designed this way, so that the user cannot see and use the Merkel Main Menu before they log in. During registration, the program asks for full name, email, and password, which generates a unique 10-digit username for the user to log in later. I do not store passwords as plain text as per coursework requirements, I hash them using std::hash<std::string> and store only the hash. This way, even if someone opens the users.csv file, they won't see actual passwords.

I also link every user to their own wallet by using the username as the key, so after login, the program loads the wallet that belongs only to that user.

(ii) Implementation logic discussion + pseudocode fragments

Where this is implemented (files/classes)

I split Task 2 into two parts: UI flows and user storage/verification.

MerkelMain.cpp

loginMenu() shows the login/register menu loop

loginUserFlow() asks for username/password and logs in

registerUserFlow() collects full name/email/password and registers

recoverUsernameFlow() verifies email then shows username

resetPasswordFlow() verifies email then updates password hash

UserManager.cpp/.h

Loads users from users.csv, registers new users, verifies login, updates password hash

User.h

Stores: username, fullName, email, passwordHash

Wallet.cpp/.h

Wallet persistence in wallets.csv using the username as the user context.

Step A: Registration flow (create account + prevent duplicates)

When I choose Register, I collect the 3 required inputs (full name, email, password).

Before saving anything, I apply two checks:

Email format validation (simple check: must include @ and a dot after it)

Duplicate prevention: I check if a user already exists with the same *full name + email* (case-insensitive), so people cannot re-register the same identity.

If it passes, I hash the password using std::hash<std::string> and store the hash (not the raw password). Generate a 10-digit numeric username and also ensure it's unique among existing users by regenerating if it clashes. And append the user record into users.csv.

```
size_t UserManager::hashPassword(const std::string &password)
{
    // Hash the password for storage/comparison
    std::hash<std::string> hasher;
    return hasher(password);
}
```

```
User u;
u.fullName = fullName;
u.email = email;
u.passwordHash = hashPassword(password);
u.username = generateUsername();

// Ensure username is unique among existing users
while (std::any_of(users.begin(), users.end(),
                   [&](const User &existing)
                   {
                       return existing.username == u.username;
                   }))
{
    u.username = generateUsername();
}

users.push_back(u);
saveUsers();
generatedUsername = u.username;
return true;
```

Finally, I link the user to their wallet by ensuring they have a wallet context saved under their username. I load it and then immediately save it, so the user exists in wallets.csv.

Step B: Login flow (verify credentials + load user wallet)

When I choose Login, I ask for username and password. Internally, the logic is that I hash the password input using the same hash function and compare it to the stored hash for that username. If it matches, login is successful and I set the program's currentUser. And immediately after login, I load that user's wallet using the username as the key, so the user sees their balances only, not another user's balances.

Step C: Forgot details support (recover username + reset password)

The coursework asks for a suitable step to remind users or reset password.

I implemented both:

Recover Username

- User enters email
- If email exists, I print the username for that account

Reset Password

- User enters email + a new password
- I hash the new password and overwrite the stored hash, then save back to users.csv
- This keeps the program usable even if a user forgets their credentials, without needing an external database system.

Pseudocode 1 - Register user

```
REGISTER-USER-FLOW():
  name ← INPUT("Full name")
  email ← INPUT("Email")
  password ← INPUT("Password")
  IF email format invalid:
    SHOW "Invalid email" and RETURN
  IF USER-EXISTS(name, email):
    SHOW "User already exists" and RETURN
  username ← GENERATE-10-DIGIT-ID()
  WHILE username already used:
    username ← GENERATE-10-DIGIT-ID()
  passHash ← HASH(password)      // std::hash<string>
  APPEND (username, name, email, passHash) TO users.csv
  LOAD-WALLET(username)          // sets wallet context
  SAVE-WALLET()                  // ensures wallet exists in wallets.csv
  SHOW username to user
```

Pseudocode 2 - Login user

```
LOGIN-USER-FLOW():
  username ← INPUT("Username")
  password ← INPUT("Password")
  passHash ← HASH(password)
  IF (username, passHash) matches a record in users.csv:
    currentUser ← that record
    isLoggedIn ← true
```

```
    LOAD-WALLET(username)
    SHOW "Login success"
ELSE:
    SHOW "Incorrect username or password"
```

(iii) Screenshots and evidence of the final output

```
PS C:\Users\ashma\OneDrive\Documents\Semester 3\OOP\midterm> ./a.exe
CSVReader::readCSV read 1021772 entries
====================== MERKLEREX LOGIN ======================
1. Login
2. Register
3. Reset Password
4. Recover Username
5. Exit
------------------------------------------------------------
Enter choice (1-5):
2
======================= REGISTER USER =======================
Full name: Ashmal Zahra
Email: ashmalz@gmail.com
Password: sandwich
[OK] Registration successful!
Your username: 1548590752
------------------------------------------------------------
====================== MERKLEREX LOGIN ======================
1. Login
2. Register
3. Reset Password
4. Recover Username
5. Exit
------------------------------------------------------------
Enter choice (1-5):
1
========================== LOGIN ============================
Username: 1548590752
Password: sandwich
====================== LOGIN SUCCESS ========================
Welcome, Ashmal Zahra!
------------------------------------------------------------
==================== WELCOME TO MERKLEREX ====================
==================== MERKLEREX MAIN MENU =====================
1: Print help
2: Print exchange stats
3: Make an offer (ask)
4: Make a bid
5: Print wallet
6: Continue to next TimeFrame
7: Show Candlestick summary
8: Wallet & Transactions Menu
9: Simulate trading activity
10: Exit program
------------------------------------------------------------
```

Register and Login Menu

```
PS C:\Users\ashma\OneDrive\Documents\Semester 3\OOP\midterm> ./a.exe
2. Register
3. Reset Password
4. Recover Username
5. Exit
------------------------------------------------------------
Enter choice (1-5):
4
====================== RECOVER USERNAME ======================
Enter your registered email: ashmal@gmail.com
[ERROR] No user found with this email.
====================== MERKLEREX LOGIN =======================
1. Login
2. Register
3. Reset Password
4. Recover Username
5. Exit
------------------------------------------------------------
Enter choice (1-5):
4
====================== RECOVER USERNAME ======================
Enter your registered email: ashmalz@gmail.com
[OK] Account found!
Your username is: 1548590752
------------------------------------------------------------
====================== MERKLEREX LOGIN =======================
1. Login
2. Register
3. Reset Password
4. Recover Username
5. Exit
------------------------------------------------------------
Enter choice (1-5):
3
======================= RESET PASSWORD =======================
Enter your registered email: ashmalz@gmail.com
Enter new password: oop-project
[OK] Password updated!
====================== MERKLEREX LOGIN =======================
1. Login
2. Register
3. Reset Password
4. Recover Username
5. Exit
------------------------------------------------------------
Enter choice (1-5):
```

Recover Username and Reset Password options in the Login Menu

TASK 3: User Wallet & Trading Transactions History

(i) Suitable explanation

In this task, I implemented a wallet and transaction history module so the program can track what the user has done financially and show useful summaries.

The user can:

1. **view their wallet balance**
2. **simulate depositing money or withdrawing money**
3. **see recent transactions** (either last 5, or all of them, or filtered by product)
4. **view activity statistics**, including how many asks/bids were placed and how much money was spent within a selected timeframe.

The main idea is that the program always stores user-related information separately in temporary CSV files, so it persists across runs and stays tied to the logged-in user.

(ii) Implementation logic discussion

Where this is implemented (files/classes)

I separated Task 3 into two parts: wallet persistence and transaction history + summaries

Wallet.cpp/.h

Stores balance in memory and persists them to wallets.csv

Loads/saves using the currently logged-in username

TransactionManager.cpp/.h

Logs transactions to transactions.csv

Shows “View Transactions” menu (last 5 / by product / all)

Computes summary statistics (overall + timeframe-based)

MerkelMain.cpp

Calls the wallet/transaction menu

Logs trading actions (bid/ask placements and matched trades) into TransactionManager

Step A: Data structure + how I persist to temporary CSV files

Wallet data

In memory, I store balances in a map<string, double> (currency → amount). I also store currentUsername inside the wallet, so saving/loading automatically targets the logged-in user.

Wallet::loadFromCSV(username) loads only rows for that username from wallets.csv.

Wallet::saveToCSV() rewrites the file in a safe way: it keeps other users' rows and replaces only the current user's rows.

```
    return;

    // Read existing file lines (if file doesn't exist yet, that's fine)
    std::ifstream in("wallets.csv");
    std::vector<std::string> lines;
    std::string line;

    if (in.is_open())
    {
        while (std::getline(in, line))
            lines.push_back(line);
        in.close();
    }

    std::ofstream out("wallets.csv", std::ios::trunc);
    if (!out.is_open())
        return;

    for (const auto &l : lines)
    {
        auto parts = CSVReader::tokenise(l, ',');
        if (parts.size() != 3)
            continue;

        // Skip old rows for the active user so we don't duplicate blocks.
        if (parts[0] == currentUsername)
            continue;

        out << l << "\n";
    }

    // Write current user's wallet entries exactly once
    if (currencies.empty())
    {
        out << currentUsername << ",USDT,0\n";
    }
    else
    {
        for (const auto &c : currencies)
            out << currentUsername << "," << c.first << "," << c.second << "\n";
    }
```

This design keeps wallet isolation simple: the user only ever sees their own balances.

Transaction data

- I created a Transaction struct containing:
    - username, type, asset, amount, value, timestamp
- I store transactions in a vector<Transaction> transactions;
- TransactionManager::logTransaction(...) pushes into the vector and appends a new row into transactions.csv.

A key decision here is that every row includes the username, so transactions.csv can hold multiple users' histories while still allowing user-scoped filtering in the UI.

```
// Log transaction (user scoped)
void TransactionManager::logTransaction(const std::string &type,
                                        const std::string &asset,
                                        double amount,
                                        double value,
                                        const std::string &timestamp)
{
    // Store username inside each row so transactions csv can hold a combined history for multiple users
    Transaction t{user.username, type, asset, amount, value, timestamp};
    transactions.push_back(t);
    saveToCSV();
}
```

Step B: Deposit / Withdraw + viewing wallet balance

I implemented these as wallet actions inside TransactionManager:

View Wallet Balance

The program prints the wallet contents in a readable table format.

Deposit Funds

The program asks the user for an amount in USDT. If the input amount is valid (> 0), I add it to the wallet and immediately save it to wallets.csv. I also log a deposit transaction into transactions.csv with the same timestamp context.

Withdraw Funds

The program shows the user how much USDT is available. It attempts to remove the amount from the wallet. If that fails (negative or exceeds balance), it prints an error. On success, it saves the wallet and logs a withdraw transaction.

Step C: Logging trading actions (orders + matched trades)

Besides deposit/withdraw, I also log trading-related actions:

When the user places an ask or bid

After the wallet check passes, the program inserts the order into the OrderBook. I log the placement as ask or bid in transactions.csv, including:

1. product (e.g., BTC/USDT)
2. amount
3. value (I record the quote-currency value of the order/trade as amount * price)

When trades are matched

When the program advances time and a sale executes for the current user, I update the wallet using wallet.processSale(sale) and log the execution. This ensures the program maintains a continuous record of user activity in the temporary CSV file.

Step D: Viewing recent transactions (user-scoped)

The View Transactions feature is implemented as a menu with three useful views:

1. **Last 5 transactions**

   The program iterates backwards through the transactions list so the newest items show first. It only displays rows where t.username == currentUser.username.

2. **Transactions by product**

   The program builds a unique list of products the user interacted with. The user selects a product, and the program prints only those rows (still user-scoped).

3. **View all transactions**

   Prints every transaction for the logged-in user.

Step E: Activity summary

It shows a complete summary of the logged-in user's activity across all recorded transactions, and it displays the overall activity window as the timeframe from the earliest user transaction timestamp to the latest user transaction timestamp.

1. Total number of asks and bids
2. A per-product breakdown (asks and bids per market pair)
3. Money spent / received is calculated and displayed in the quote currency of each market pair using the stored value = amount * price
    - If the market is X/USDT, totals contribute to USDT spent/received.
    - If the market is X/BTC, totals contribute to BTC spent/received.
4. Deposits/withdrawals are tracked separately (USDT only) and shown alongside trading totals.

ETH/DOGE appear in the wallet as base assets acquired or sold, but money spent/received is intentionally reported only in the quote currencies used by the dataset's market pairs (USDT and BTC).

(iii) Screenshots and evidence of the final output

```
-------------------------------------------------------------
==================== WELCOME TO MERKLEREX ====================
==================== MERKLEREX MAIN MENU ====================
1: Print help
2: Print exchange stats
3: Make an offer (ask)
4: Make a bid
5: Print wallet
6: Continue to next TimeFrame
7: Show Candlestick summary
8: Wallet & Transactions Menu
9: Simulate trading activity
10: Exit program
-------------------------------------------------------------
Current time is: 2020/06/01 11:57:30
-------------------------------------------------------------
Enter choice (1-10):
8
==================== WALLET & TRANSACTIONS ====================
1. View Wallet Balance
2. Deposit Funds
3. Withdraw Funds
4. View Transactions
5. Activity Summary
6. Back
-------------------------------------------------------------
Enter choice (1-6):
```

Choose option 8 to view Wallet and Transactions History

Choose option 2 to Deposit Funds and option 1 to view wallet balance

```
=================== WALLET & TRANSACTIONS ===================
1. View Wallet Balance
2. Deposit Funds
3. Withdraw Funds
4. View Transactions
5. Activity Summary
6. Back
------------------------------------------------------------
Enter choice (1-6):
2
======================= DEPOSIT FUNDS =======================
Amount (USDT): 5000
[OK] Deposit successful
=================== WALLET & TRANSACTIONS ===================
1. View Wallet Balance
2. Deposit Funds
3. Withdraw Funds
4. View Transactions
5. Activity Summary
6. Back
------------------------------------------------------------
Enter choice (1-6):
1
============================ WALLET ============================
USDT : 5000.000000
------------------------------------------------------------
```

```
==================== MERKLEREX MAIN MENU ====================
1: Print help
2: Print exchange stats
3: Make an offer (ask)
4: Make a bid
5: Print wallet
6: Continue to next TimeFrame
7: Show Candlestick summary
8: Wallet & Transactions Menu
9: Simulate trading activity
10: Exit program
------------------------------------------------------------
Current time is: 2020/06/01 11:57:30
------------------------------------------------------------
Enter choice (1-10):
4
====================== CREATE BID ORDER ======================
Select a product:
1: BTC/USDT
2: DOGE/BTC
3: DOGE/USDT
4: ETH/BTC
5: ETH/USDT
------------------------------------------------------------
Enter choice (1-5):
3
Enter price: 10
Enter amount: 10
------------------------------------------------------------
Checking wallet...
Wallet looks good.
======================= ORDER SUMMARY =======================
Type:    BID
Product: DOGE/USDT
Price:   10.00000000
Amount:  10.00000000
Time:    2020/06/01 11:57:30
------------------------------------------------------------
[OK] Order created successfully!
------------------------------------------------------------
```

Choose option 4 back in the main menu to place a bid

```
PS C:\Users\ashma\OneDrive\Documents\Semester 3\OOP\midterm> ./a.exe
[OK] Order created successfully!
------------------------------------------------------------
==================== MERKLEREX MAIN MENU ====================
1: Print help
2: Print exchange stats
3: Make an offer (ask)
4: Make a bid
5: Print wallet
6: Continue to next TimeFrame
7: Show Candlestick summary
8: Wallet & Transactions Menu
9: Simulate trading activity
10: Exit program
------------------------------------------------------------
Current time is: 2020/06/01 11:57:30
------------------------------------------------------------
Enter choice (1-10):
6
================= ADVANCE TO NEXT TIMEFRAME =================
Processing market activity at time:
2020/06/01 11:57:30
------------------------------------------------------------
Product: BTC/USDT
Matched trades: 0
------------------------------------------------------------
Product: DOGE/BTC
Matched trades: 0
------------------------------------------------------------
Product: DOGE/USDT
Matched trades: 1
  - Price: 0.00257729 | Amount: 10.00000000
------------------------------------------------------------
Product: ETH/BTC
Matched trades: 0
------------------------------------------------------------
Product: ETH/USDT
Matched trades: 0
------------------------------------------------------------
[OK] Timeframe advanced successfully.
------------------------------------------------------------
Current time is now: 2020/06/01 11:57:35
------------------------------------------------------------
```

Bid matched and sale processed after choosing option 6 for moving onto the next timeframe

```
------------------------------------------------------------
Enter choice (1-10):
8
=================== WALLET & TRANSACTIONS ===================
1. View Wallet Balance
2. Deposit Funds
3. Withdraw Funds
4. View Transactions
5. Activity Summary
6. Back
------------------------------------------------------------
Enter choice (1-6):
5
====================== ACTIVITY SUMMARY ======================
TIMEFRAME: 2020/06/01 11:57:30 -> 2020/06/01 11:57:30
------------------------------------------------------------
Asks placed: 0
Bids placed: 1
Total money spent: 0.025773 USDT
Total money received: 0 USDT
Net change from trading: -0.025773 USDT
------------------------------------------------------------
Deposits: 1 (total 5000.00000000 USDT)
Withdrawals: 0 (total 0.00000000 USDT)
------------------------------------------------------------
----- Per Product -----
DOGE/USDT: 1 bids, 0 asks, spent 0.025773 USDT, received 0.000000 USDT
------------------------------------------------------------
=================== WALLET & TRANSACTIONS ===================
1. View Wallet Balance
2. Deposit Funds
3. Withdraw Funds
4. View Transactions
5. Activity Summary
6. Back
------------------------------------------------------------
Enter choice (1-6):
1
========================== WALLET ==========================
DOGE : 10.000000
USDT : 4999.974227
------------------------------------------------------------
```

Activity Summary back in the ‘Wallet and Transactions Menu’ reflects the bid made

TASK 4: Simulating User Trading Activities

(i) Suitable explanation

In this task, I added a trading simulation so the user can generate fresh ask/bid activity using the current system timestamp, and the program can update the user's wallet and persist those new transactions. This is important because the provided market dataset ends years earlier, so without simulation the platform would feel stuck in the past and the user would have very little recent activity to work with.

The reason I designed the simulation to span multiple days is that it makes the output more meaningful for analysis tools that depend on calendar grouping. When the user views daily candlesticks after simulation, the table shows multiple daily rows instead of a single one. This is not random filler; it's a deliberate way to produce a realistic-looking timeline so the program can demonstrate daily grouping, date-range filtering, and activity over time in a way a single-day burst cannot.

Finally, I made the simulated bid/ask prices realistic by deriving them from market context and applying a small spread. In simple terms: I derived a reference price from current market data and applied a small bid–ask spread (around ±0.1%) so bids and asks form believable quotes; OHLC is still computed normally using the first/last price in each period. This also prevents ASK and BID candlestick series from collapsing into identical values.

(ii) Implementation logic discussion + pseudocode fragments

Where this is implemented (files/classes)

MerkelMain::simulateTradingActivity()

Runs the simulation loop, creates new orders using current timestamps, matches them, updates wallet, and logs transactions.

TransactionManager.cpp/.h

Persists simulated actions into transactions.csv (append style) and prevents accidental duplicate simulation runs for the same user.

Wallet.cpp/.h

Ensures simulated buys/sells actually affect balances and are saved to wallets.csv.

MerkelMain::showCandlesticks() merges market orders with user trade history by calling transactionManager.getTradesAsOrderBookEntries(currentUser.username) and adding those entries into the candle input set.

```
// Candlestick view
void MerkelMain::showCandlesticks()
{
    while (true)
    {
        // Product selection
        auto products = orderBook.getKnownProducts();
        UI::printTitle("AVAILABLE PRODUCTS");

        for (int i = 0; i < (int)products.size(); i++)
        {
            std::cout << (i + 1) << ": " << products[i] << std::endl;
        }

        UI::printDivider();
        int pChoice = UI::readIntInRange(1, products.size());
        std::string product = products[pChoice - 1];

        // Gather orders
        std::vector<OrderBookEntry> all = orderBook.getAllOrders();
        // Include user trade history so candles reflect executed trades too
        auto userTrades = transactionManager.getTradesAsOrderBookEntries(currentUser.username);
        all.insert(all.end(), userTrades.begin(), userTrades.end());

        std::vector<OrderBookEntry> asks, bids;

        for (auto &e : all)
        {
            if (e.product == product)
            {
                if (e.orderType == OrderBookType::ask)
                    asks.push_back(e);
                else if (e.orderType == OrderBookType::bid)
                    bids.push_back(e);
            }
        }

        // Default yearly summary
        auto yearlyAsks = CandlestickCalculator::computeYearlyCandles(asks);
        auto yearlyBids = CandlestickCalculator::computeYearlyCandles(bids);
```

TransactionManager::getTradesAsOrderBookEntries() converts each saved simulated transaction into synthetic BID + ASK entries using price = value/amount plus a 0.1% spread, specifically so the candle series remains meaningful (ASK and BID don't become identical).

Step A: I avoid multiple simulation runs unless the user confirms

At the start of simulateTradingActivity(), I check whether simulation-like transactions already exist for the logged-in user. If yes, the program warns the user and asks for confirmation before running again. This prevents cluttering transactions.csv with repeated simulation blocks just because the menu option was clicked twice.

Step B: I generate timestamps across multiple days (deliberate design)

I set:

- daysToGenerate = 5
- ticksPerDay = 2

So, for each product, the program creates activity over five different calendar days, with two different times per day. I generate each day by taking the system date as the base, then adding day offsets (day 0 → day 4). Within the day, I offset seconds, so the two ticks are not the same timestamp.

This produces multiple daily candles, so the daily candlestick view and date-range filtering are actually demonstrated properly. It makes the simulated trade history look like realistic continued usage over a few days instead of one artificial burst.

Step C: How I calculate suitable prices (reference price + trend + noise + spread)

For each product, I anchor simulated prices to real market context from the dataset at the program's current timeframe. I take a reference level marketNow from the order book at currentTime (lowest ask if available, otherwise highest bid), and I also sample marketNext at the next timeframe to estimate a short-term trend.

I compute trend = (marketNext − marketNow) / marketNow and clamp it to avoid unrealistic jumps.
For each simulated day/tick, I calculate a mid-price:

- mid = marketNow × (1 + dayDrift + noise)
  where dayDrift is trend-based drift across the 5 generated days, and noise is a small random percentage to prevent flat lines.

Then I build a realistic bid/ask around the mid-price using a small spread:

- marketBid = mid − (mid × spreadPct)
- marketAsk = mid + (mid × spreadPct)

Finally, to ensure the simulated user order actually matches, I place the counterparty order slightly better by around 0.1%. If the user is buying: user BID is placed slightly above marketAsk and a dataset ASK is placed at marketAsk, so the bid crosses the ask and matches. If the user is selling: user ASK is placed at marketBid and a dataset BID is placed slightly above it, so the bid crosses the ask and matches.

Step D: Date gap handling (2022 → 2025/2026)

To manage the large gap between the last record in the dataset and the current system timestamp, I treat simulation as new activity on top of the historical dataset, not as something that must fill in the missing years. Every simulated order/transaction is stamped using the current system timestamp, so it naturally appears in 2025.

Both the transaction history and candlestick summaries work by grouping and filtering based on timestamps. As a result, historical dataset records remain grouped under their own older dates/years, and simulated records appear separately under 2025 bucket, without mixing the time periods or overwriting the dataset timeline. In simple terms, I keep the dataset as historical market data and append simulation as current user activity.

Step E: I keep the simulation wallet-safe (no negative balances)

During simulation, I deliberately prevent the wallet from ever going negative by resizing or skipping trades before inserting them into the order book.

For each simulated tick, I first decide whether to attempt a buy or a sell (the simulator alternates buy/sell across ticks). Then I read the user's current balances for that product's currencies:

A BUY requires the quote currency (e.g., BTC/USDT requires USDT; ETH/BTC requires BTC).

A SELL requires the base currency (e.g., BTC/USDT requires BTC).

If the simulator wants to buy but the user has no quote balance, and the user can sell (has base), I switch to a sell instead and vice versa. If the user has neither currency available, I skip that attempt.

When a trade direction is possible, I compute the maximum safe size from the wallet:

For BUY, the maximum affordable base amount is quoteBalance / execPrice.

For SELL, the maximum sellable base amount is baseBalance.

If the intended amount is larger than what the wallet can support, I do not fail the simulation; instead, I shrink the amount down to the maximum safe amount by applying a small * 0.999 buffer. This is why the console may print messages like “insufficient BTC/USDT” which means the requested order size was too large and was resized to fit the user’s available balance.

After placing the user order and the matching dataset-side order, I match them and only update balances for sales attributed to the logged-in user by calling wallet.processSale(sale). Finally, I save the wallet to CSV. This combination of pre-check, resize/skip and user-attributed wallet updates ensure the simulation produces activity while keeping balances realistic and never negative.

Step F: I persist all simulated activity to the transaction CSV and link it into candlesticks

Every simulated placement is logged via transactionManager.logTransaction(…), which appends to transactions.csv. Later, when the user opens Candlestick Summary, the program reads those saved simulated records (user-scoped), converts them into candle input entries with a tiny synthetic spread, and the daily candlestick view naturally shows multiple days in 2025.

Pseudocode – Simulate Trading Activity

```
SIMULATE-TRADING():
  IF user already has simulated trades:
    ask user to confirm; if no → return
  products ← all known products
  FOR each product IN products:
    marketNow  ← reference price from current timeframe
    marketNext ← reference price from next timeframe
    trend ← clamp((marketNext - marketNow)/marketNow)
    FOR day = 0..4:
      tsDay ← today + day
      FOR tick = 0..1:
        ts ← tsDay + small time offset
        mid ← marketNow * (1 + trend drift + small noise)
        (marketBid, marketAsk) ← mid ± small spread


        choose buy/sell based on wallet balances
        adjust amount down if needed
        place user order + matching dataset order (±0.1% to guarantee match)
        match orders, update wallet on user sale
        append transaction row to transactions.csv
  save wallet to wallets.csv
```

(iii) Screenshots and evidence of the final output

```
PS C:\Users\ashma\OneDrive\Documents\Semester 3\OOP\midterm> ./a.exe
==================== MERKLEREX MAIN MENU ====================
1: Print help
2: Print exchange stats
3: Make an offer (ask)
4: Make a bid
5: Print wallet
6: Continue to next TimeFrame
7: Show Candlestick summary
8: Wallet & Transactions Menu
9: Simulate trading activity
10: Exit program
------------------------------------------------------------
Current time is: 2020/06/01 11:57:35
------------------------------------------------------------
Enter choice (1-10):
9
================ SIMULATING TRADING ACTIVITY ================
[ADJUST] BTC/USDT SELL at 2025/12/23 08:00:19 resized from 0.00540456 to 0.00074593 (insufficient BTC)
[ADJUST] BTC/USDT SELL at 2025/12/25 08:00:19 resized from 0.00761742 to 0.00564591 (insufficient BTC)
------------------------------------------------------------
Product BTC/USDT -> Executed: 10 | Adjusted: 2 | Skipped: 0
------------------------------------------------------------
[ADJUST] DOGE/USDT SELL at 2025/12/22 08:00:36 resized from 191.78874620 to 159.80178629 (insufficient DOGE)
[ADJUST] DOGE/USDT SELL at 2025/12/23 08:00:19 resized from 167.90245874 to 0.15980179 (insufficient DOGE)
[ADJUST] DOGE/USDT SELL at 2025/12/25 08:00:19 resized from 207.68249173 to 108.98863682 (insufficient DOGE)
------------------------------------------------------------
Product DOGE/USDT -> Executed: 10 | Adjusted: 3 | Skipped: 0
------------------------------------------------------------
[ADJUST] ETH/USDT SELL at 2025/12/22 08:00:36 resized from 0.11629488 to 0.09688673 (insufficient ETH)
[ADJUST] ETH/USDT SELL at 2025/12/23 08:00:19 resized from 0.13817468 to 0.00009689 (insufficient ETH)
[ADJUST] ETH/USDT SELL at 2025/12/25 08:00:19 resized from 0.17267024 to 0.11141433 (insufficient ETH)
------------------------------------------------------------
Product ETH/USDT -> Executed: 10 | Adjusted: 3 | Skipped: 0
------------------------------------------------------------
------------------------------------------------------------
Product DOGE/BTC -> Executed: 10 | Adjusted: 0 | Skipped: 0
------------------------------------------------------------
[ADJUST] ETH/BTC SELL at 2025/12/23 08:00:19 resized from 0.12609587 to 0.10844913 (insufficient ETH)
[ADJUST] ETH/BTC SELL at 2025/12/25 08:00:19 resized from 0.10368273 to 0.09805002 (insufficient ETH)
------------------------------------------------------------
Product ETH/BTC -> Executed: 10 | Adjusted: 2 | Skipped: 0
------------------------------------------------------------
[OK] Trading simulation completed successfully.
Summary: Executed trades: 50 | Adjusted orders: 10 | Skipped attempts: 0
==================== MERKLEREX MAIN MENU ====================
```

Choose option 9 to simulate trading activity

View all Transactions option

```
PS C:\Users\ashma\OneDrive\Documents\Semester 3\OOP\midterm> ./a.exe
------------------------------------------------------------
Enter choice (1-4):
3
TIMESTAMP            TYPE        ASSET           AMOUNT          VALUE
------------------------------------------------------------
2025/12/26 08:00:36  ask         ETH/BTC         0.18057230      0.00450968
2025/12/26 08:00:19  bid         ETH/BTC         0.15011808      0.00372001
2025/12/25 08:00:36  bid         ETH/BTC         0.13256921      0.00330401
2025/12/25 08:00:19  ask         ETH/BTC         0.09795197      0.00243944
2025/12/24 08:00:36  ask         ETH/BTC         0.16659115      0.00411865
2025/12/24 08:00:19  bid         ETH/BTC         0.14686716      0.00365445
2025/12/23 08:00:36  bid         ETH/BTC         0.11766556      0.00294363
2025/12/23 08:00:19  ask         ETH/BTC         0.10834069      0.00268709
2025/12/22 08:00:36  ask         ETH/BTC         0.12282973      0.00306531
2025/12/22 08:00:19  bid         ETH/BTC         0.13089670      0.00324650
2025/12/26 08:00:36  ask         DOGE/BTC        267.84987284    0.00007197
2025/12/26 08:00:19  bid         DOGE/BTC        288.48631822    0.00007805
2025/12/25 08:00:36  bid         DOGE/BTC        296.73712218    0.00007994
2025/12/25 08:00:19  ask         DOGE/BTC        239.58407731    0.00006480
2025/12/24 08:00:36  ask         DOGE/BTC        229.68034579    0.00006220
2025/12/24 08:00:19  bid         DOGE/BTC        194.37684156    0.00005236
2025/12/23 08:00:36  bid         DOGE/BTC        238.89286180    0.00006473
2025/12/23 08:00:19  ask         DOGE/BTC        178.31687881    0.00004791
2025/12/22 08:00:36  ask         DOGE/BTC        136.41981877    0.00003677
2025/12/22 08:00:19  bid         DOGE/BTC        177.86750766    0.00004819
2025/12/26 08:00:36  ask         ETH/USDT        0.15375984      36.51255497
2025/12/26 08:00:19  bid         ETH/USDT        0.13772194      32.70859322
2025/12/25 08:00:36  bid         ETH/USDT        0.11630866      27.62006966
2025/12/25 08:00:19  ask         ETH/USDT        0.11130292      26.24634298
2025/12/24 08:00:36  ask         ETH/USDT        0.18069318      42.86243190
2025/12/24 08:00:19  bid         ETH/USDT        0.14160521      33.48880269
2025/12/23 08:00:36  bid         ETH/USDT        0.15050220      35.85198729
2025/12/23 08:00:19  ask         ETH/USDT        0.00009679      0.02306177
2025/12/22 08:00:36  ask         ETH/USDT        0.09678984      22.88658779
2025/12/22 08:00:19  bid         ETH/USDT        0.09688673      23.15955548
2025/12/26 08:00:36  ask         DOGE/USDT       212.14784588    0.54465521
2025/12/26 08:00:19  bid         DOGE/USDT       196.24845329    0.50718982
2025/12/25 08:00:36  bid         DOGE/USDT       253.25454184    0.65389666
2025/12/25 08:00:19  ask         DOGE/USDT       108.87964818    0.28004299
2025/12/24 08:00:36  ask         DOGE/USDT       243.91854357    0.62541245
2025/12/24 08:00:19  bid         DOGE/USDT       166.07166180    0.43058943
2025/12/23 08:00:36  bid         DOGE/USDT       186.83535879    0.48274461
2025/12/23 08:00:19  ask         DOGE/USDT       0.15964198      0.00040909
2025/12/22 08:00:36  ask         DOGE/USDT       159.64198450    0.41127622
2025/12/22 08:00:19  bid         DOGE/USDT       149.80178629    0.38552560
```

```
------------------------------------------------------------
[OK] Trading simulation completed successfully.
Summary: Executed trades: 50 | Adjusted orders: 10 | Skipped attempts: 0
==================== MERKLEREX MAIN MENU ====================
1: Print help
2: Print exchange stats
3: Make an offer (ask)
4: Make a bid
5: Print wallet
6: Continue to next TimeFrame
7: Show Candlestick summary
8: Wallet & Transactions Menu
9: Simulate trading activity
10: Exit program
------------------------------------------------------------
Current time is: 2020/06/01 11:57:35
------------------------------------------------------------
Enter choice (1-10):
5
============================ WALLET ============================
BTC : 0.007285
DOGE : 381.973796
ETH : 0.102213
USDT : 4904.611271

------------------------------------------------------------
```

Walle updated and reflects the trading activity

```
PS C:\Users\ashma\OneDrive\Documents\Semester 3\OOP\midterm> ./a.exe
3: Make an offer (ask)
4: Make a bid
5: Print wallet
6: Continue to next TimeFrame
7: Show Candlestick summary
8: Wallet & Transactions Menu
9: Simulate trading activity
10: Exit program
------------------------------------------------------------
Current time is: 2020/06/01 11:57:35
------------------------------------------------------------
Enter choice (1-10):
7
==================== AVAILABLE PRODUCTS ====================
1: BTC/USDT
2: DOGE/BTC
3: DOGE/USDT
4: ETH/BTC
5: ETH/USDT
------------------------------------------------------------
Enter choice (1-5):
1
================ YEARLY SUMMARY (BTC/USDT) ================
ASK Candlesticks (Yearly)
------------------------------------------------------------
Year        Open              High              Low               Close
2020        9549.3999982900   9614.9238645400   9495.9293918700   9600.0000000000
2025        9589.0907938198   9589.0907938198   9486.6175805626   9534.0139984781

BID Candlesticks (Yearly)
------------------------------------------------------------
Year        Open              High              Low               Close
2020        9503.4480656300   9588.5205724100   9452.9401565700   9542.8826887600
2025        9598.6798846136   9608.2785644982   9505.6098079511   9553.1011135901
------------------------------------------------------------
```

Candlestick summary reflects the trading activity

Monthly and Daily Summaries

```
PS C:\Users\ashma\OneDrive\Documents\Semester 3\OOP\midterm> ./a.exe
2
================ MONTHLY SUMMARY (BTC/USDT) ================
ASK (Monthly)
------------------------------------------------------------
Month        Open             High             Low              Close
2020-06      9549.3999982900  9614.9238645400  9495.9293918700  9600.0000000000
2025-12      9589.0907938198  9589.0907938198  9486.6175805626  9534.0139984781

BID (Monthly)
------------------------------------------------------------
Month        Open             High             Low              Close
2020-06      9503.4480656300  9588.5205724100  9452.9401565700  9542.8826887600
2025-12      9598.6798846136  9608.2785644982  9505.6098079511  9553.1011135901
------------------------------------------------------------
Additional Options:
1. View Daily Summary
2. View Monthly Summary
3. View Yearly Summary
4. View Summary by Date Range
5. Change Product
6. Exit Candlestick View
------------------------------------------------------------
Enter choice (1-6):
1
================= DAILY SUMMARY (BTC/USDT) =================
ASK (Daily)
------------------------------------------------------------
Date         Open             High             Low              Close
2020-06-01   9549.3999982900  9614.9238645400  9495.9293918700  9600.0000000000
2025-12-22   9589.0907938198  9589.0907938198  9573.0346223091  9573.0346223091
2025-12-23   9496.1136942568  9536.5573984712  9486.6175805626  9536.5478619138
2025-12-24   9584.9658808128  9584.9658808128  9499.4932888735  9499.4932888735
2025-12-25   9578.7818842993  9586.0766262958  9569.2031024150  9586.0670402191
2025-12-26   9502.7764147307  9543.5575560341  9502.7669119543  9534.0139984781

BID (Daily)
------------------------------------------------------------
Date         Open             High             Low              Close
2020-06-01   9503.4480656300  9588.5205724100  9452.9401565700  9542.8826887600
2025-12-22   9598.6798846136  9608.2785644982  9592.1998567882  9592.1998567882
2025-12-23   9505.6098079511  9555.6400498256  9505.6098079511  9555.6400498256
2025-12-24   9594.5508466936  9604.1453975403  9518.5112934558  9518.5112934558
2025-12-25   9588.3606661836  9605.2583656250  9588.3606661836  9605.2583656250
2025-12-26   9512.2791911455  9553.1011135901  9512.2791911455  9553.1011135901
------------------------------------------------------------
```

Ln 2, Col 1 Spaces: 4 UTF-8 CRLF {} Plain Text

```
------------------------------------------------------------
Additional Options:
1. View Daily Summary
2. View Monthly Summary
3. View Yearly Summary
4. View Summary by Date Range
5. Change Product
6. Exit Candlestick View
------------------------------------------------------------
Enter choice (1-6):
5
==================== AVAILABLE PRODUCTS ====================
1: BTC/USDT
2: DOGE/BTC
3: DOGE/USDT
4: ETH/BTC
5: ETH/USDT
------------------------------------------------------------
Enter choice (1-5):
4
================= YEARLY SUMMARY (ETH/BTC) =================
ASK Candlesticks (Yearly)
------------------------------------------------------------
Year         Open             High             Low              Close
2020         0.0251094500     0.0252832500     0.0247586200     0.0251611100
2025         0.0247772354     0.0249919166     0.0246983653     0.0249493918

BID Candlesticks (Yearly)
------------------------------------------------------------
Year         Open             High             Low              Close
2020         0.0246194800     0.0250842500     0.0245344800     0.0249468600
2025         0.0248020126     0.0250419254     0.0247478115     0.0249993405
------------------------------------------------------------
```

Change Product to view its summary

```
------------------------------------------------------------
Additional Options:
1. View Daily Summary
2. View Monthly Summary
3. View Yearly Summary
4. View Summary by Date Range
5. Change Product
6. Exit Candlestick View
------------------------------------------------------------
Enter choice (1-6):
4
==================== DATE RANGE FILTER ====================
Available range: [2020-06-01 - 2025-12-26]
------------------------------------------------------------
Enter start date (YYYY-MM-DD):
2020-06-01
Enter end date (YYYY-MM-DD):
2025-12-23
================= FILTERED RANGE (ETH/BTC) =================
ASK (Filtered)
------------------------------------------------------------
Date          Open            High            Low             Close
2020-06-01    0.0251094500    0.0252832500    0.0247586200    0.0251611100
2025-12-22    0.0247772354    0.0249557518    0.0247772106    0.0249307961
2025-12-23    0.0248022448    0.0249919166    0.0247774425    0.0249918916

BID (Filtered)
------------------------------------------------------------
Date          Open            High            Low             Close
2020-06-01    0.0246194800    0.0250842500    0.0245344800    0.0249468600
2025-12-22    0.0248020126    0.0249807076    0.0248020126    0.0249807076
2025-12-23    0.0248270470    0.0250419254    0.0248270470    0.0250419254
------------------------------------------------------------
```

Summary by Date Range

```
==================== WALLET & TRANSACTIONS ====================
1. View Wallet Balance
2. Deposit Funds
3. Withdraw Funds
4. View Transactions
5. Activity Summary
6. Back
------------------------------------------------------------
Enter choice (1-6):
5
====================== ACTIVITY SUMMARY ======================
TIMEFRAME: 2020/06/01 11:57:30 -> 2025/12/29 22:20:39
------------------------------------------------------------
Asks placed: 25
Bids placed: 26
Total money spent: 489.096110 USDT | 0.01617632 BTC
Total money received: 372.251784 USDT | 0.01598448 BTC
Net change from trading: -116.844326 USDT | -0.00019184 BTC
------------------------------------------------------------
Deposits: 1 (total 5000 USDT)
Withdrawals: 0 (total 0 USDT)
------------------------------------------------------------
----- Per Product -----
BTC/USDT: 5 bids, 5 asks, spent 326.652800 USDT, received 253.312820 USDT
DOGE/BTC: 5 bids, 5 asks, spent 0.00025242 BTC, received 0.00024454 BTC
DOGE/USDT: 6 bids, 5 asks, spent 2.354610 USDT, received 1.978165 USDT
ETH/BTC: 5 bids, 5 asks, spent 0.01592390 BTC, received 0.01573994 BTC
ETH/USDT: 5 bids, 5 asks, spent 160.088700 USDT, received 116.960799 USDT
------------------------------------------------------------
```

Activity Summary reflects the trading activity

```
==================== MERKLEREX MAIN MENU ====================
1: Print help
2: Print exchange stats
3: Make an offer (ask)
4: Make a bid
5: Print wallet
6: Continue to next TimeFrame
7: Show Candlestick summary
8: Wallet & Transactions Menu
9: Simulate trading activity
10: Exit program
------------------------------------------------------------
Current time is: 2020/06/01 11:57:35
------------------------------------------------------------
Enter choice (1-10):
9
================ SIMULATING TRADING ACTIVITY ================
Trading simulation has already been executed for this user.
Do you want to run the simulation again? (y/n): n
[ERROR] Simulation cancelled by user.
==================== MERKLEREX MAIN MENU ====================
```

Confirmation for another trading activity

TASK 5: Printing user interaction menu to support user login & trading operations

(i) Suitable explanation

For this task, I focused on making the whole program feel safe to use from the console. Since the trading platform is fully menu-driven from login menu to main menu then sub-menus like wallet/transactions, the biggest risk is bad input: typing letters when a number is expected, entering a menu choice outside the valid range, or entering invalid numeric amounts.

So, the goal of my Task 5 work is that wrong input never crashes the program or breaks the input stream. Instead, the program should show a clear error message and let the user try again, while keeping navigation consistent across menus.

(ii) Implementation logic discussion + pseudocode fragments

Where this is implemented (files/classes)

This task is implemented using a mix of centralised input helpers and menu loops:

UIHelpers.cpp/.h

Contains reusable input and display functions, including:

UI::readIntInRange(min, max) (range-checked menu input)

UI::readString(prompt) (safe line input)

UI::readDouble() (safe numeric input without breaking cin)

MerkelMain.cpp

loginMenu() keeps the user inside the login screen until valid login happens.

init() runs the main menu loop and reads options safely.

processUserOption() routes the user's menu choice to the correct feature.

TransactionManager.cpp

showMenu() and showTransactions() are also loop-driven menus and use the same safe input approach.

depositFunds() / withdrawFunds() read amounts using UI::readDouble() and validate the values.

This structure matters because I only had to solve input robustness once inside UIHelpers and then reuse it everywhere.

Key measures I used to verify and validate user inputs:

Step A: Range-checked menu input which prevents invalid choices and prevents stream failure

Whenever the program expects a menu choice, I do not read a raw integer and trust it. I use UI::readIntInRange(min, max), which:

1. keeps prompting until the user enters a valid integer,
2. rejects out-of-range numbers (e.g., choice 99 in a 1–10 menu),
3. clears the cin failure state and discards the rest of the line so the input stream doesn't get "stuck".

This prevents abrupt termination and prevents buggy follow-up reads.

Pseudocode:

```
READ-INT-IN-RANGE(min, max):
  LOOP forever:
    PROMPT "Enter choice (min-max):"
    IF canReadInteger(value) AND min ≤ value ≤ max:
      DISCARD rest of current input line
      RETURN value
    PRINT "Invalid input"
    CLEAR input fail state
    DISCARD rest of current input line
```

Step B: Safe reading of doubles which prevents cin from breaking when the user types text

For prices, amounts, deposits, etc., I used UI::readDouble(). Instead of relying on cin >> double which can leave the stream in a failed state if the user types "abc", it reads a full line and converts using stod, inside a try/catch loop.

That means the user can type something invalid and the program still continues normally, it simply prints an error and asks again.

Pseudocode:

```
READ-DOUBLE():
  LOOP forever:
    line ← READ-LINE()
    TRY:
      RETURN toDouble(line)
    CATCH:
      PRINT "Invalid number. Try again:"
```

Step C: Consistent menu loops (the user is never thrown out due to a mistake)
Instead of single-shot menus that assume perfect input, I used loops:

MerkelMain::loginMenu() repeats until login succeeds or the user exits.

MerkelMain::init() repeats the main menu until the user chooses Exit.

TransactionManager::showMenu() repeats the wallet/transactions menu until Back.

This ensures that incorrect input does not end the program or randomly jump the user out of a feature.

Step D: Domain validation beyond parsing (inputs can be valid types but still invalid actions)
Even if the user enters a correct number, it might not be a valid action. So, in addition to type checking, I also validate meaning, for withdrawing funds, the wallet removal function is checked, and an error is printed if the withdrawal is invalid e.g., more than available.

This prevents situations where inputs are syntactically valid but logically impossible.

(iii) Screenshots and evidence of the final output

```
=================== MERKLEREX MAIN MENU ===================
1: Print help
2: Print exchange stats
3: Make an offer (ask)
4: Make a bid
5: Print wallet
6: Continue to next TimeFrame
7: Show Candlestick summary
8: Wallet & Transactions Menu
9: Simulate trading activity
10: Exit program
------------------------------------------------------------
Current time is: 2020/06/01 11:57:35
------------------------------------------------------------
Enter choice (1-10):
0
[ERROR] Invalid input. Enter number between 1 and 10
Enter choice (1-10):
8
================== WALLET & TRANSACTIONS ==================
1. View Wallet Balance
2. Deposit Funds
3. Withdraw Funds
4. View Transactions
5. Activity Summary
6. Back
------------------------------------------------------------
Enter choice (1-6):
9
[ERROR] Invalid input. Enter number between 1 and 6
Enter choice (1-6):
2
====================== DEPOSIT FUNDS ======================
Amount (USDT): abc
[ERROR] Invalid number. Try again:
200
[OK] Deposit successful
=================== WALLET & TRANSACTIONS ==================
```

```
=================== WALLET & TRANSACTIONS ===================
1. View Wallet Balance
2. Deposit Funds
3. Withdraw Funds
4. View Transactions
5. Activity Summary
6. Back
-------------------------------------------------------------
Enter choice (1-6):
3
======================= WITHDRAW FUNDS =======================
Available: 5199.97422710 USDT
Amount: 5250
[ERROR] Invalid withdrawal
```

Invalid Inputs rejected gracefully